Table of Contents

Software Requirements Specification

Field	Value

Field	Value
Document ID	NOVA-SRS-001
Version	0.1
Status	Draft
Owner	Laxmi Poudel
Date	2026-09-08
Conforms to	ISO/IEC/IEEE 29148:2018, clause 9.6

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for NOVA, a locally hosted agent that converts software requirements into structured test plans and retains user corrections as durable memory. It is the authoritative requirements baseline for the initial release and the source against which verification is performed.

Intended audience: the implementing engineer, and any reviewer assessing the completeness or testability of the requirement set.

1.2 Scope

The software product is named **NOVA**.

NOVA accepts a natural-language software requirement, classifies it by type, and produces a structured test plan conforming to a versioned schema. It records user corrections at test case granularity, extracts durable rules from those corrections subject to explicit user confirmation, and applies them to subsequent structurally similar requirements. It exposes a complete execution trace for every task and measures whether corrections recur.

NOVA does not execute tests, generate executable test code, ingest source repositories, or integrate with issue trackers. It performs no model training of any kind.

Benefits and objectives are stated in NOVA-VS-001.

1.3 Product overview

1.3.1 Product perspective

NOVA is a self-contained product with no predecessor system. It consists of four deployable components: a browser-based user interface, a REST API service, an asynchronous worker process, and a relational datastore with vector indexing. Inference is supplied by an external runtime resident on the host, addressed through an internal provider abstraction.

External interfaces are limited to the inference runtime. The system has no other outbound network dependency in its default configuration.

1.3.2 Product functions

Function	Summary
Requirement intake	Accept, validate, classify, and persist a submitted requirement
Test plan generation	Produce a schema-conformant test plan using constrained decoding
Memory retrieval	Select applicable stored rules and expose the selection scores
Strategy selection	Choose a generation playbook from recorded outcome scores
Quality evaluation	Apply deterministic checks and a bounded model-based review before returning results
Feedback capture	Record accept, edit, reject, and addition events at test case granularity
Memory extraction	Derive candidate rules from corrections and present them for confirmation
Memory management	Browse, edit, pin, and delete stored rules
Execution tracing	Persist and present a complete record of each task execution
Evaluation	Execute a versioned dataset, compute metrics, and gate configuration changes

1.3.3 User characteristics

Class	Description
Primary user	Software or test engineer, two to six years of experience, fluent in test design and comfortable with command-line tooling and HTTP APIs. Holds specific conventions regarding case format, mandatory case types, and priority taxonomy. Professionally disinclined to accept unexplained automated output
Operator	The same individual. Responsible for deployment, model provisioning, and backup

There is no administrative user class in the initial release.

1.3.4 Limitations

ID	Limitation	Origin
CON-1	Development effort limited to approximately 130 to 180 hours	Resource availability
CON-2	Single engineer, no external code review	Project structure
CON-3	Inference constrained to a 6 GB VRAM budget	Measured hardware
CON-4	Deployment is local only	Project decision
CON-5	Synthetic and open-source data only	Project decision
CON-6	No model fine-tuning	Scope boundary

1.4 Definitions, acronyms, and abbreviations

Term	Definition
AC	Acceptance criterion. A verifiable condition stated within a requirement
ADR	Architecture Decision Record
Applied lesson	A stored memory retrieved and injected into a generation prompt for a given task
Candidate memory	An extracted rule awaiting user confirmation. Not yet retrievable
Constrained decoding	Generation restricted at the token level to output conforming to a supplied schema
CRR	Correction Recurrence Rate. See section 3.1.9
Execution trace	The ordered, persisted record of every step, model call, and score for one task
Memory	A durable rule derived from a user correction, with provenance, scope, and confidence
Playbook	A named, versioned generation strategy comprising required case types, a prompt fragment, and check configuration
Provenance	The originating task, correction, and timestamp recorded against a memory
Tenant	The isolation boundary for all user-owned data
Test plan	The structured output of one generation, comprising a summary and an ordered set of test cases
Tombstone	The residual record of a deleted memory, retaining identifiers and timestamps but no content

2. References

Reference	Title
NOVA-VS-001	Vision and Scope
NOVA-SAD-001	Software Architecture Document
NOVA-STP-001	Software Test Plan
NOVA-TM-001	Threat Model
ISO/IEC/IEEE 29148:2018	Systems and software engineering, life cycle processes, requirements engineering
ISO/IEC/IEEE 29119-3:2021	Software testing, test documentation
RFC 9457	Problem Details for HTTP APIs
arc42 v9	Architecture documentation template

3. Specific requirements

Requirements use `shall` for mandatory behaviour. Each is uniquely identified, individually verifiable, and traced in section 4.

Priority values follow MoSCoW: **M** must, **S** should, **C** could, **W** will not.

3.1 Functions

3.1.1 Requirement intake

ID	Requirement	Pri
FR-1	The system shall accept a requirement consisting of natural-language text, a requirement type, and optional supplementary context	M
FR-1.1	The system shall reject a requirement whose text is empty or exceeds the configured length limit, returning a message identifying the violated constraint	M
FR-1.2	The system shall persist every accepted requirement together with its submission timestamp and the identifier of the configuration version in force at submission	M

3.1.2 Test plan generation

ID	Requirement	Pri
FR-2	The system shall generate a test plan conforming to the versioned test plan schema, using constrained decoding to enforce structural validity	M
FR-2.1	The system shall record the schema version against every generated plan and shall remain able to read plans produced under earlier schema versions	M
FR-2.2	Where a generated plan fails schema validation, the system shall attempt a bounded number of repair retries before recording the task as failed	M
FR-2.3	The system shall record every generation attempt, whether successful or not, against the validity metric	M
FR-2.4	Where the assembled prompt would exceed the available context budget, the system shall reduce retrieved memory content before reducing requirement content	M

3.1.3 Strategy selection

ID	Requirement	Pri
FR-3	The system shall select one playbook per task from at least two available playbooks, using recorded outcome scores	M
FR-3.1	The system shall apply bounded random exploration at a configurable rate with a non-zero lower bound	M
FR-3.2	The system shall apply a static default playbook where the observation count for a requirement type is below the configured minimum	M
FR-3.3	The system shall constrain each outcome score update to a bounded step and shall clamp scores to the interval [-1, +1]	M

ID	Requirement	Pri
FR-3.4	The system shall record whether a given selection was exploratory, and shall present that fact in the user interface	M
FR-3.5	The system shall be able to recompute all outcome scores from the recorded feedback event history	M

3.1.4 Memory retrieval

ID	Requirement	Pri
FR-4	The system shall retrieve stored memories ranked by a weighted combination of semantic similarity, recency, and confidence	M
FR-4.1	The system shall apply requirement type and tenant as filters prior to ranking	M
FR-4.2	The system shall return memories marked as pinned and in scope irrespective of their rank	M
FR-4.3	The system shall exclude memories scoring below the configured relevance threshold, and shall record the exclusion	M
FR-4.4	The system shall present each applied memory to the user together with its component and combined scores	M
FR-4.5	Where retrieval returns no memory above threshold, the system shall proceed using playbook defaults and shall state that no prior lesson was applied	M
FR-4.6	Where the embedding service is unavailable, the system shall fail the task with an explicit cause and shall not proceed with an empty retrieval result	M

3.1.5 Feedback capture

ID	Requirement	Pri
FR-5	The system shall record accept, edit, reject, and addition events against individual test cases	M
FR-5.1	The system shall accept an optional freeform correction against a task	M
FR-5.2	The system shall retain the originally generated text of an edited case alongside the edited text	M
FR-5.3	The system shall normalize each feedback event to a score in the interval [-1, +1] using a versioned weighting scheme	M
FR-5.4	The system shall treat feedback events as immutable once written	M

3.1.6 Memory extraction and confirmation

ID	Requirement	Pri
FR-6	The system shall derive candidate rules from freeform corrections	M

ID	Requirement	Pri
FR-6.1	The system shall reject a candidate that is not generalizable, is not actionable, claims a scope broader than its evidence, exceeds the configured length limit, or is empty of semantic content	M
FR-6.2	The system shall flag for review any candidate whose text is shaped as an instruction directed at the system, and shall not store such a candidate without explicit user action	M
FR-6.3	The system shall not store any memory without explicit user confirmation	M
FR-6.4	The system shall detect near-duplicate candidates and shall reinforce the existing memory rather than creating a second record	M
FR-6.5	The system shall detect a candidate that contradicts an existing in-scope memory and shall present both to the user for resolution	M
FR-6.6	Where extraction fails, the system shall retain the original correction text for later processing	M
FR-6.7	The system shall record provenance against every stored memory, identifying the originating task, correction, and timestamp	M

3.1.7 Memory management

ID	Requirement	Pri
FR-7	The system shall allow the user to browse stored memories, filtered by scope, status, and pinned state	M
FR-7.1	The system shall allow the user to edit the text, scope, and pinned state of a stored memory	M
FR-7.2	The system shall re-derive the embedding and reset confidence to its initial value when memory text is edited	M
FR-7.3	The system shall allow the user to delete a stored memory, removing its content and rendering it unretrievable	M
FR-7.4	The system shall retain the identifier, timestamps, and tenant of a deleted memory so that historical traces remain coherent	M
FR-7.5	The system shall reduce the confidence of a memory that remains unapplied over time, and shall archive rather than delete a memory falling below the configured floor	M
FR-7.6	The system shall link a superseding memory to the memory it replaces and shall retain the superseded record	M
FR-7.7	The system shall export the complete memory set in a machine-readable format	S

3.1.8 Execution tracing

ID	Requirement	Pri

ID	Requirement	Pri
FR-8	The system shall persist an execution trace for every task, comprising the playbook selection and rationale, all retrieval results with scores, every model call, and every deterministic check verdict	M
FR-8.1	The system shall record retrieval scores for candidate memories that were evaluated but not selected	M
FR-8.2	The system shall record prompt token count, completion token count, latency, attempt number, and validity outcome for each model call	M
FR-8.3	The system shall write trace steps as they complete, such that a task failing part-way retains the trace of all completed steps	M
FR-8.4	The system shall store a cryptographic digest of each prompt in place of the prompt text	M

3.1.9 Evaluation and configuration control

ID	Requirement	Pri
FR-9	The system shall compute and persist the metrics defined in NOVA-STP-001 for each task and each evaluation run	M
FR-9.1	The system shall execute an evaluation suite against a content-hash-versioned dataset and report per-metric aggregates	M
FR-9.2	The system shall compute the Correction Recurrence Rate as the proportion of previously confirmed corrections that reappear as errors on later structurally similar tasks	M
FR-10	The system shall maintain versioned prompt and configuration records	M
FR-10.1	The system shall reject activation of a configuration version that is not linked to a passing evaluation run, and shall enforce this at the data layer	M
FR-10.2	The system shall retain superseded configuration versions	M

3.1.10 Inference provider management

ID	Requirement	Pri
FR-11	The system shall support multiple inference providers behind a common interface, defaulting to local execution	M
FR-11.1	The system shall change the active provider only on explicit user action, and shall never substitute a provider automatically	M
FR-11.2	The system shall display the active provider in the user interface	M
FR-11.3	The system shall report inference provider reachability through its readiness endpoint	M
FR-	Where a required model is absent, the system shall return a message naming the model	M

ID	Requirement	Pri
11.4	and the action required to obtain it	

3.1.11 Presentation

ID	Requirement	Pri
FR-12	The system shall display accumulated token count and elapsed duration for each completed task	S
FR-13	The system shall export a test plan in Markdown and comma-separated value formats	C
FR-14	The system shall present two executions of the same requirement in a comparison view	C
FR-15	The system shall support memory sharing between users	W

3.2 Performance requirements

Targets marked *provisional* are set from judgment and are subject to revision once the corresponding measurement exists. Where a measurement exists it is cited.

ID	Requirement	Target	Basis
NFR-1	Structured output shall be schema-valid on first attempt	â‰¥98%	Provisional. 12 of 12 observed over a 12-generation sample. See spike
NFR-2	End-to-end task duration, 95th percentile	â‰¤90 s	Provisional and known to be slack. Generation alone measured 9 to 26 s warm on the reference configuration
NFR-3	Memory retrieval duration, 95th percentile	â‰¤200 ms	Provisional
NFR-4	Evaluation suite total duration	â‰¤15 min	Provisional. Cold model load is a material contributor and shall be reported separately
NFR-5	Inference provider call failure rate	â‰¤2%	Provisional

Reference configuration for all timing requirements: NVIDIA RTX 4050 Laptop, 6 GB VRAM, 4B-parameter model at 4-bit quantization, 8192-token context. Timing requirements are void without this qualification.

3.3 Usability requirements

ID	Requirement	Pri
NFR-6	The user interface shall remain responsive to interaction while a task is executing	M

ID	Requirement	Pri
NFR-7	The system shall present task status such that a queued, running, failed, or cancelled task is distinguishable without ambiguity	M
NFR-8	The system shall permit cancellation of a running task, retaining the partial trace	M
NFR-9	Error messages presented to the user shall identify the failing step and the corrective action where one exists	M

3.4 Interface requirements

3.4.1 User interfaces

Four views: task submission and results, execution trace, memory management, and evaluation results. Rendered in a current desktop browser. No mobile layout is provided.

3.4.2 Application programming interfaces

ID	Requirement	Pri
NFR-10	The system shall expose a versioned HTTP interface under a path prefix, admitting only additive change within a version	M
NFR-11	The system shall return errors in the format defined by RFC 9457	M
NFR-12	The system shall honour an idempotency key on task creation, returning the original task identifier on replay	M
NFR-13	The system shall paginate collection responses using opaque cursors	M
NFR-14	The system shall publish an interface description generated from its request and response models, and shall fail its build where the published description diverges from the generated one	M

3.4.3 External interfaces

The inference runtime is addressed over HTTP on the host loopback interface. The system supplies a JSON Schema with each generation request and receives a schema-conformant completion together with token and timing telemetry.

3.5 System operations

3.5.1 Nominal operation

1. The user submits a requirement with its type.
2. The system validates the submission, persists it, enqueues a job, and returns a task identifier without waiting for execution.

3. A worker claims the job, reads the active configuration version, selects a playbook, retrieves applicable memories, generates a plan under schema constraint, applies deterministic checks and a bounded model-based review, and persists the plan and trace.
4. The user reviews the plan alongside the applied lessons and their scores.
5. The user records accept, edit, or reject against individual cases, and may supply a freeform correction.
6. The system derives candidate rules and presents them for confirmation.
7. On confirmation, the system embeds and stores each rule.
8. On a subsequent structurally similar requirement, the stored rules are retrieved and applied, and are shown as applied lessons.

3.5.2 Exception operation

ID	Condition	Required behaviour
OP-1	Memory store empty	Generate from playbook defaults. State explicitly that no prior lesson was applied
OP-2	Generated output fails schema validation	Bounded repair retry, then terminal failure with a stated cause. All attempts recorded
OP-3	Inference provider unreachable	Terminal failure with a stated cause. Previously completed results remain accessible
OP-4	Retrieved memories conflict	Present the conflict. Do not resolve it silently
OP-5	Requirement text contains an instruction directed at the system	Treat as data. Neutralize and record
OP-6	All cases rejected	Record as a strongly negative outcome. Prompt for a freeform correction
OP-7	Retrieval returns nothing above threshold	Proceed on defaults. Record the empty result
OP-8	A memory in current use is deleted	Honour the deletion immediately and completely. Retain a tombstone reference in historical traces
OP-9	Task exceeds its duration budget	Remain cancellable. Preserve the partial trace
OP-10	Later feedback contradicts an earlier correction	Apply recency and confidence weighting. Supersede the earlier rule. Do not average
OP-11	Worker process terminates during execution	Reclaim the job after lease expiry. Preserve the partial trace
OP-12	Datastore unreachable	Fail fast. Perform no partial writes

3.6 System modes and states

Task states and their permitted transitions:

State	Permitted successors
`queued`	`running`, `cancelled`
`running`	`completed`, `failed`, `cancelled`
`completed`	terminal
`failed`	terminal
`cancelled`	terminal

Memory states:

State	Permitted successors
`candidate`	`active`, discarded
`active`	`superseded`, `archived`, `deleted`
`archived`	`active`, `deleted`
`superseded`	`deleted`
`deleted`	terminal

Configuration version states: `draft`, `evaluated`, `active`, `retired`. Transition to `active` requires an associated passing evaluation run.

3.7 Physical characteristics

Not applicable. NOVA is a software-only product with no hardware deliverable.

3.8 Environmental conditions

The system operates on a single workstation running Windows 11 with a container runtime and a GPU-accelerated inference runtime resident on the host. No requirement is placed on temperature, humidity, shock, or electromagnetic environment.

3.9 System security

ID	Requirement	Pri
NFR-15	The system shall associate every user-owned record with a tenant identifier	M
NFR-16	The system shall apply tenant scoping at the data access layer such that no query path can omit it	M
NFR-17	The system shall resolve the tenant identifier server-side and shall never accept it from a client-supplied value	M
NFR-18	The system shall permit zero cross-tenant data disclosure. This requirement admits no tolerance	M
NFR-	The system shall treat requirement text, correction text, and model output as untrusted	M

ID	Requirement	Pri
19	data, delimited and labelled within prompts and never concatenated into instruction context	
NFR-20	The system shall provide the inference model with no capability to read files, execute commands, or initiate network requests	M
NFR-21	The system shall validate all model output against its schema before persistence or presentation	M
NFR-22	The system shall escape untrusted content on presentation	M
NFR-23	The system shall exclude requirement text, correction text, and memory content from application logs by default	M
NFR-24	The system shall bound every retry loop and shall define a terminal state for each	M
NFR-25	The system shall hold no credential in its default configuration	M

Threat analysis is in NOVA-TM-001.

3.10 Information management

ID	Requirement	Pri
NFR-26	The system shall treat feedback events as the authoritative record and shall derive all outcome scores from them	M
NFR-27	The system shall retain feedback events independently of the lifecycle of any memory derived from them	M
NFR-28	The system shall version feedback normalization weights such that historical scores can be recomputed under a revised scheme	M
NFR-29	The system shall treat execution traces as append-only and immutable once written	M
NFR-30	The system shall version the evaluation dataset by content hash and shall refuse to compare runs across dataset versions	M
NFR-31	The system shall support a documented backup and restore procedure	M

3.11 Policies and regulations

No regulatory regime applies. The system processes synthetic and open-source data only and transmits no data beyond the host in its default configuration. Personally identifiable information detection is not implemented; this is a recorded gap, and its status changes should authentic data enter the system.

3.12 System life cycle sustainment

ID	Requirement	Pri
NFR-32	The system shall start from a clean checkout by a single documented command	M
NFR-33	The system shall apply schema changes through forward-only versioned migrations	M
NFR-34	The system shall reproduce a prior execution given the same input, random seed, model version, and configuration version	M
NFR-35	Automated tests covering memory, retrieval, scoring, and validation logic shall achieve at least 80% statement coverage	M

3.13 Packaging, handling, shipping, and transportation

Not applicable.

4. Verification

Each requirement is verified by one of: **T** automated test, **D** demonstration, **A** analysis or inspection, **M** measurement against the evaluation dataset.

Requirement group	Method	Verification artifact
FR-1 to FR-2.4	T, M	Schema conformance tests; structured output validity metric
FR-3 to FR-3.5	T, D	Seeded selection tests; score recomputation test; poisoning scenario
FR-4 to FR-4.6	T, M	Ranking unit tests; retrieval precision against a labelled relevance set; cross-tenant isolation suite
FR-5 to FR-5.4	T	Normalization tests across all event types; immutability test
FR-6 to FR-6.7	T, A, M	Validation rejection tests; adversarial injection-into-memory scenarios; extraction precision measured against hand-labelled corrections
FR-7 to FR-7.7	T, D	Deletion completeness test; tombstone audit; lifecycle transition tests
FR-8 to FR-8.4	T	Trace completeness test on a deliberately failed task; log content assertion
FR-9 to FR-10.2	T, M	Harness self-test; regression gate demonstration on a deliberately degraded configuration
FR-11 to FR-11.4	T, D	Provider contract suite; explicit switch test; readiness endpoint test
NFR-1 to NFR-5	M	Instrumented measurement on the stated reference configuration
NFR-6 to NFR-9	D	Demonstration and exploratory session
NFR-10 to NFR-14	T	Interface description drift test; idempotency test; pagination tests

Requirement group	Method	Verification artifact
NFR-15 to NFR-25	T, A	Cross-tenant isolation suite as an unconditional gate; adversarial suite; log content assertion; architecture inspection
NFR-26 to NFR-31	T, D	Score recomputation test; restore drill
NFR-32 to NFR-35	D, T	Clean-checkout deployment on a separate host; reproducibility check; coverage gate

Acceptance is complete when every requirement of priority **M** has passed its stated verification method and the results are recorded.

Appendix A. Assumptions and dependencies

ID	Assumption	Status
ASM-1	Local inference sustains schema fidelity at acceptable latency	Confirmed. 12 of 12 valid, 9 to 26 s warm, fully GPU-resident
ASM-2	A local model extracts reusable rules from freeform corrections at acceptable precision	Unverified. Highest project uncertainty. Fallback is user-authored rules with model assistance
ASM-3	Structural similarity between requirements is definable with sufficient stability to support FR-9.2	Unverified
ASM-4	Deterministic checks capture sufficient quality signal to serve as a gate	Unverified
ASM-5	Inference nondeterminism remains within evaluation gate margins under a fixed seed	Unverified

ID	Dependency	Status
DEP-1	Inference runtime, host-resident	Installed
DEP-2	Model weights	Five candidates retrieved
DEP-3	Relational datastore with vector indexing	Not provisioned
DEP-4	Container runtime	Installed, not running
DEP-5	Embedding model, and consequently the vector dimension	Not selected. Blocks schema definition

Appendix B. Acronyms and abbreviations

Abbreviation	Expansion
AC	Acceptance criterion
ADR	Architecture Decision Record
API	Application Programming Interface
CI	Continuous Integration
CRR	Correction Recurrence Rate
CSV	Comma-Separated Values
FR	Functional Requirement
NFR	Non-Functional Requirement
MoSCoW	Must, Should, Could, Will not
REST	Representational State Transfer
SAD	Software Architecture Document
SRS	Software Requirements Specification
VRAM	Video Random Access Memory

Revision history

Version	Date	Author	Change
0.1	2026-09-08	Laxmi Poudel	Initial draft